EVENTHUB

Plataforma de Gestão de Eventos em Microsserviços Poliglotas

EQUIPE

Lucas Aguiar · Marx Haron · Vitor Fontenele

Ian Domingos · Marcos Aurélio

MAPA DA APRESENTAÇÃO

Sumário

Seis perspectivas: do domínio de negócio à validação experimental.

01

Introdução

Cenário de domínio e pergunta orientadora

02

Estratégia de Decomposição

DDD, fronteiras e granularidade

03

Arquitetura & Padrões

C4, Gateway, AMQP e Circuit Breakers

04

Software na Prática

Jornada integrada e SPA Vue 3

05

Validação Arquitetural

Concorrência e injeção de falhas

06

Conclusão

Benefícios, custos e trade-offs

01

Introdução

O contexto do problema e suas forças arquiteturais

Cenário do Domínio & Pergunta Central

O desafio do EventHub

DOMÍNIO

Gestão de congressos, seminários e minicursos para múltiplas organizações.

DOIS ATORES CENTRAIS

Participante: inscrição, pagamento, presença e certificado.
Organizador: eventos, vagas, check-in e relatórios.

RQ02 · CARGA ASSIMÉTRICA

20× mais acessos na abertura de inscrições; certificados operam quase ociosos no pós-evento.

Pergunta central & forças

"Which business capabilities should become independently deployable services, and why?"

RN01 / RQ04 · CONCORRÊNCIA

Zero overbooking em atividades disputadas.

RN05 / RQ03 · FALHAS

Avisos ou relatórios indisponíveis não bloqueiam reservas.

RQ01 · EVOLUÇÃO

Alterar canais e templates sem reimplantar o núcleo.

02

Estratégia de Decomposição e Microsserviços

Capacidades de negócio como limites de autonomia

Limites de Contexto e Stack Poliglota

6 microsserviços candidatos, separados por domínio, taxa de mudança e perfil operacional.

Núcleo transacional

JAVA 21

- **user-service**
  IAM · JWT, BCrypt e papéis ORGANIZER / PARTICIPANT
- **event-service**
  Eventos e atividades; capacidade, check-in e certificados
- **registration-service**
  Vagas, bloqueio pessimista ACID, espera e relatórios

I/O & integrações

PYTHON 3.12

- **search-service**
  Catálogo e saldo dinâmico de vagas · RF04 / RQ02
- **payment-service**
  Checkout e provedor simulado: aprovação, recusa e expiração
- **notification-service**
  Consumidor AMQP, fallback REST e falhas controladas · RN05

POSTGRESQL 16 · BANCO ÚNICO · SINGLE WRITER POR TABELA · ADR-02

03

Arquitetura e Decisões de Design

Como os serviços se conectam e resistem a falhas

Topologia Integrada: Norte–Sul vs. Leste–Oeste



Borda Norte–Sul

GATEWAY

NGINX 1.27 ALPINE
Entrada única em :8080 e proxy reverso /api/*

SPA + SWAGGER
Vue 3 e /swagger.html sob mesma origem; sem CORS

REDE & RASTREIO
Rede eventhub-net, JWT e X-Correlation-ID

Comunicação Leste–Oeste

HÍBRIDA

RABBITMQ 3.13
Topic exchange eventhub.domain.events

AFTER_COMMIT + DLQ
Publicação após commit; retenção em eventhub.notifications.dlq

CIRCUIT BREAKERS
Resilience4j · PyBreaker + Tenacity, com fallback REST

Architectural Decision Records (ADRs)

Cinco decisões explícitas que orientam os trade-offs do EventHub.

Fundações arquiteturais

ADR 01—03

- **ADR-01 · API Gateway**
  Nginx unifica tráfego Norte–Sul e roteia os contratos REST.
- **ADR-02 · Persistência**
  PostgreSQL compartilhado com propriedade lógica; Single Writer.
- **ADR-03 · Concorrência**
  SELECT … FOR UPDATE para preservar invariantes de capacidade.

Poliglota & resiliência

ADR 04—05

- **ADR-04 · Stack poliglota**
  Java/Spring Boot para transações densas; Python/FastAPI para leitura e I/O.
- **ADR-05 · Comunicação híbrida**
  Pub/Sub RabbitMQ, DLQ, Circuit Breaker; broker-first com fallback HTTP.

04

Software na Prática

Da descoberta ao certificado validável

EVENTHUB · MICROSSERVIÇOS POLIGLOTAS

Jornada Ponta a Ponta da Inscrição

Um fluxo de negócio atravessando os limites dos microsserviços.

Da reserva ao pagamento

ETAPAS 01—03

- **01 · DESCOBERTA**
  Participante consulta o catálogo e saldo no search-service.
- **02 · RESERVA**
  registration-service executa FOR UPDATE, cria PENDING_PAYMENT e libera lock.
- **03 · CHECKOUT**
  payment-service simula aprovação ou recusa pela interface.

Da confirmação ao certificado

ETAPAS 04—06

- **04 · WEBHOOK**
  Aprovação confirma a vaga; recusa libera ou promove o 1º da WAITLIST.
- **05 · CHECK-IN**
  Organizador registra presença sem duplicidade via event-service.
- **06 · CERTIFICADO**
  Somente após o evento e presença comprovada, com validação pública · RN06.

05

Validação Arquitetural

Concorrência, injeção de falhas e evidências

Validação Experimental de Resiliência & Concorrência

Cenários e resultados informados pela equipe do projeto.

Cenários críticos testados

- **RN01 · CONCORRÊNCIA**
  Requisições paralelas sobre vagas limitadas: zero overbooking; excedentes entram em WAITLIST.

- **RN05 / RQ03 · FALHA HTTP 503**
  Notification-service indisponível: transação principal confirmada, sem bloqueio.

- **RN03 / RN04 · PAGAMENTO RECUSADO**
  Compensação desocupa a vaga e permite a promoção da lista de espera.

Evidências de resiliência

100%

retenção na DLQ
durante falha do worker

< 1 ms

resposta fail-fast
com circuito aberto

FILA: eventhub.notifications.dlq

06

Conclusão & Reflexão Crítica

O valor do desacoplamento frente ao custo da distribuição

Balanço Crítico da Distribuição

“What did we gain — and what did we pay for distributing the system?”

O que ganhamos

BENEFÍCIOS

- **ESCALABILIDADE SELETIVA**
  Busca e inscrições escalam sem onerar eventos.
- **ISOLAMENTO DE FALHAS**
  Falhas periféricas não derrubam catálogo ou autenticação.
- **ESPECIALIZAÇÃO**
  JPA/locks no Java; Async I/O no Python.
- **EVOLUÇÃO DESACOPLADA**
  Templates e canais mudam sem re-deploy geral.

O que pagamos

CUSTOS

- **OPERAÇÃO DISTRIBUÍDA**
  Containers, RabbitMQ, rede e monitoramento.
- **CONSISTÊNCIA EVENTUAL**
  Notificações chegam após confirmação transacional.
- **RASTREABILIDADE**
  IDs de correlação e contratos REST/AMQP.
- **ACOPLAMENTO DE DADOS**
  Banco único reduz autonomia e impõe governança.

Obrigado!

Dúvidas e Demonstração Prática

REPOSITÓRIO

github.com/marxharon/eventhub

SWAGGER

localhost:8080/swagger.html

CHAOS LAB · localhost:8080 · Painel Organizador